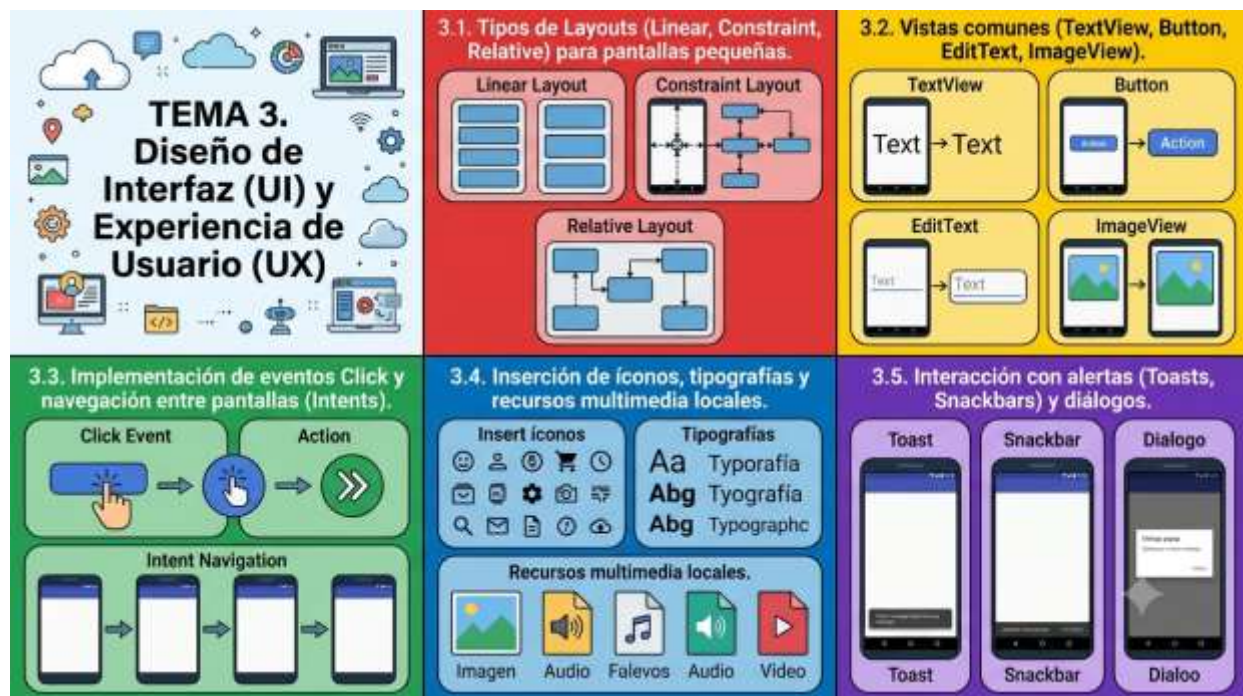


TEMA N° 3



TEMA 3 DISEÑO DE INTERFAZ (UI) Y EXPERIENCIA DE USUARIO (UX)



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Dominar la arquitectura visual móvil**, comprendiendo la diferencia crítica entre la interfaz de usuario (UI) y la experiencia de usuario (UX) para construir aplicaciones eficientes que respondan a las necesidades reales de nuestro entorno local.
2. **Construir pantallas totalmente adaptables y fluidas**, utilizando los diferentes tipos de Layouts y componentes interactivos del SDK de Android, asegurando que la aplicación se visualice correctamente tanto en teléfonos de última generación como en dispositivos antiguos de gama baja.
3. **Implementar flujos de navegación interactivos y sistemas de retroalimentación**, conectando múltiples pantallas mediante Intents y comunicando estados de forma clara al usuario a través de alertas, diálogos y recursos multimedia personalizados.



PRÁCTICA



PREGUNTA DETONANTE

¿Por qué una aplicación móvil técnicamente perfecta y sin errores de código puede terminar siendo un total fracaso si los usuarios de nuestro municipio no logran comprender dónde hacer clic para realizar una acción o se cansan de esperar a que carguen las imágenes?

EL DESAFÍO EN EL CONTEXTO DE PAILÓN

Imaginen la siguiente situación real dentro de nuestra comunidad: un grupo de emprendedores del **Barrio Central Fátima**, dedicados al comercio formal y actividades parroquiales, junto con transportistas del **Barrio 24 de Septiembre** (asentados cerca de la estación de tren y los campos deportivos), han solicitado al **CEA Herman Ortiz Camargo** el desarrollo de una aplicación móvil llamada "*Pailón Conectado*". El objetivo de esta app es permitir a los vivientes de barrios alejados como **San Expedito**, **Las Misiones** o **Oto Waver** consultar los horarios de transporte, hacer pedidos de productos de la canasta básica y ver la disponibilidad de canchas o eventos en el coliseo del **Barrio 27 de Mayo**.

Los estudiantes del semestre anterior desarrollaron la aplicación e instalaron el servidor en el nuevo módulo educativo del CEA en el **Barrio Saó**. Sin embargo, ha surgido un grave problema: los comerciantes de la tercera edad en el Barrio Central Fátima no pueden leer las letras porque son muy pequeñas, los mototaxistas que operan bajo el sol radiante de Pailón no logran distinguir los botones debido al bajo contraste de color, y los usuarios de **Jardines de Pailón** o **San José** cierran la aplicación porque esta se congela al intentar cargar imágenes pesadas a través de conexiones de datos móviles inestables. La aplicación funciona por dentro, pero su interfaz es confusa y frustrante. Como futuros **Técnicos en Sistemas Informáticos**, nuestro deber es rediseñar por completo esta aplicación, aplicando principios profesionales de UI y UX para salvar el proyecto y garantizar que sea útil para cada vecino de Pailón.



TEORÍA



3.1. TIPOS DE LAYOUTS (LINEAR, CONSTRAINT, RELATIVE) PARA PANTALLAS PEQUEÑAS

En el desarrollo de aplicaciones móviles, un **Layout** (o contenedor de vistas) es un archivo XML que define la estructura visual de una pantalla. Dado que en nuestro contexto local muchos usuarios poseen teléfonos con pantallas pequeñas o de resoluciones antiguas, elegir el contenedor adecuado es crucial para evitar que los elementos se superpongan o queden fuera del área visible.

A) *LinearLayout (Diseño Lineal)*

Organiza a sus componentes hijos de forma secuencial, uno detrás de otro, ya sea en una fila horizontal o en una columna vertical mediante el atributo `android:orientation`.

1. **Caso de uso:** Es ideal para formularios simples de registro o pantallas de inicio de sesión, como el acceso para los comerciantes del Barrio Central Fátima.
2. **Atributo clave:** `android:layout_weight` (peso). Permite distribuir el espacio disponible de manera proporcional entre las vistas, asegurando que se adapten a cualquier ancho o alto de pantalla.

XML

```
<LinearLayout      xmlns:android="
http:
//schemas.android.com/apk/res/android"      android:layout_width="
match_parent"      android:layout_height="
match_parent"      android:orientation="
vertical"      android:padding="16dp"> </LinearLayout>
```

B) *RelativeLayout (Diseño Relativo)*

Permite posicionar los elementos en relación con el contenedor padre o con respecto a otros elementos hermanos (por ejemplo: "colocar el botón A debajo del texto B").



1. **Desventaja en pantallas pequeñas:** Aunque es flexible, si se anidan demasiados contenedores, el procesador del teléfono trabaja el doble para calcular las posiciones, ralentizando la aplicación en dispositivos de gama baja.

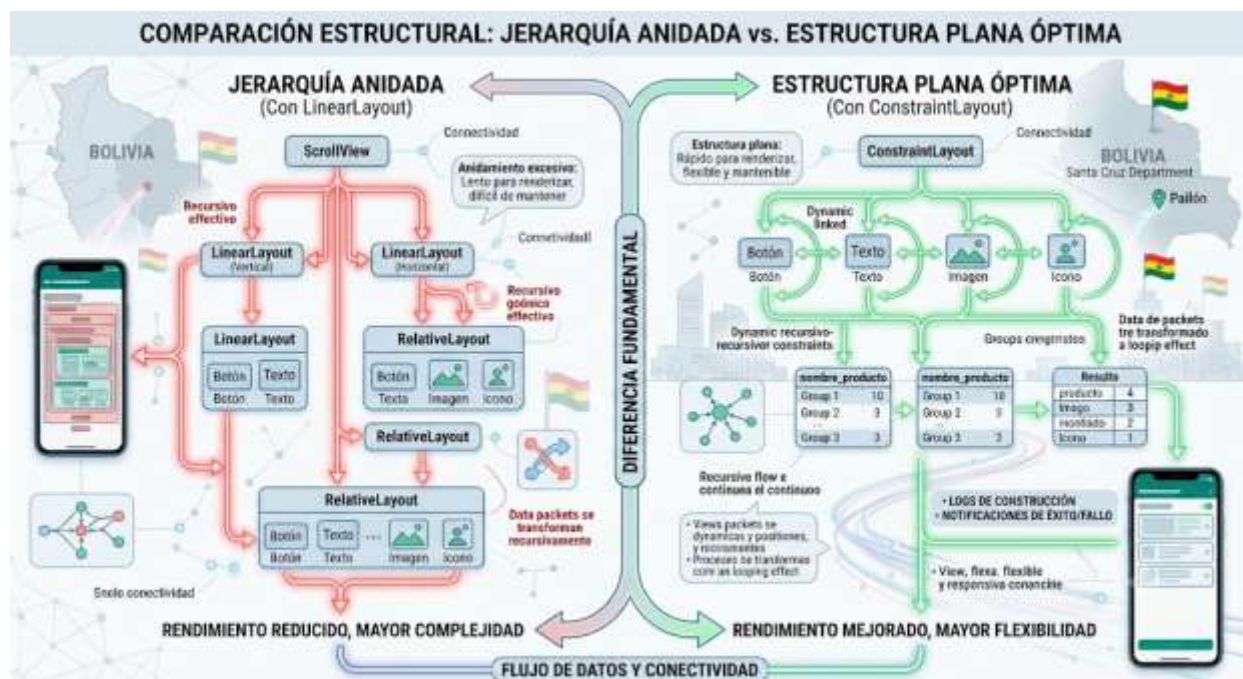
C) ConstraintLayout (Diseño de Restricciones)

Es el contenedor moderno más potente de Android. Permite crear layouts grandes y complejos con una jerarquía de vistas plana (sin contenedores anidados). Funciona mediante "anclas" o restricciones que sujetan cada lado de una vista a otra vista o al borde de la pantalla.

1. **Caso de uso:** Es obligatorio para interfaces complejas, garantizando una adaptabilidad matemática perfecta en cualquier tamaño de pantalla.

XML

```
<androidx.constraintlayout.widget.ConstraintLayout    xmlns:android="
http://schemas.android.com/apk/res/android"    xmlns:app="
http://schemas.android.com/apk/res-auto"    android:layout_width="
match_parent"    android:layout_height="
match_parent">    </androidx.constraintlayout.widget.ConstraintLayout>
```





3.2. VISTAS COMUNES (TEXTVIEW, BUTTON, EDITTEXT, IMAGEVIEW)

Las vistas (Views) son los componentes visuales individuales con los que interactúa el usuario. Para asegurar una UX excelente en el CEA y en los barrios de Pailón, debemos configurarlas usando las unidades de medida correctas: **dp** (densidad de píxeles independientes) para tamaños de componentes y **sp** (píxeles independientes de escala) para textos, permitiendo que el sistema agrande la letra si el usuario lo tiene configurado así por problemas visuales.

Componentes Fundamentales:

1. **TextView:** Se utiliza exclusivamente para mostrar textos estáticos o dinámicos en la pantalla.
2. **EditText:** Es un campo de entrada de texto que permite al usuario introducir datos (nombres, contraseñas, números de teléfono). Es vital configurar el atributo `android:inputType` para desplegar el teclado correcto (numérico para teléfonos, texto para direcciones).
3. **Button:** Un componente interactivo sobre el cual el usuario hace clic para ejecutar una acción concreta.
4. **ImageView:** Diseñado para renderizar gráficos, fotos o íconos vectoriales locales.

XML

```
<androidx.constraintlayout.widget.ConstraintLayout    xmlns:android="
http:
//schemas.android.com/apk/res/android"    xmlns:app="
http:
//schemas.android.com/apk/res-auto"    android:layout_width="
match_parent"    android:layout_height="
match_parent"    android:padding="20dp">    <ImageView
android:id="@+id/imgLogoReporte"    android:layout_width="100dp"
android:layout_height="100dp"    android:src="@drawable/ic_parque_verde"
android:contentDescription="Ícono de parque ecológico"
app:layout_constraintTop_toTopOf="
parent"    app:layout_constraintStart_toStartOf="
parent"    app:layout_constraintEnd_toEndOf="
parent" />    <TextView    android:id="@+id/txtTituloFormulario"
android:layout_width="
wrap_content"    android:layout_height="
```




```

wrap_content"            android:text="
Reporte de Parques - Barrio Residencial Norte"            android:textSize="16sp"
android:textStyle="
bold"            android:textColor="
#2E7D32" android:layout_marginTop="16dp"
app:layout_constraintTop_toBottomOf="@id/imgLogoReporte"
app:layout_constraintStart_toStartOf="
parent"            app:layout_constraintEnd_toEndOf="
parent" />    <EditText            android:id="@+id/edtDetalleProblema"
android:layout_width="0dp" android:layout_height="
wrap_content"            android:hint="
Ej. Limpieza de maleza en parque infantil" android:inputType="
textMultiLine" android:layout_marginTop="12dp"
app:layout_constraintTop_toBottomOf="@id/txtTituloFormulario"
app:layout_constraintStart_toStartOf="
parent"            app:layout_constraintEnd_toEndOf="
parent" />    <Button            android:id="@+id/btnEnviarReporte"
android:layout_width="0dp"            android:layout_height="
wrap_content"            android:text="
ENVIAR ALERTA COMUNITARIA"            android:backgroundTint="
#2E7D32"            android:textColor="
#FFFFFF"            android:layout_marginTop="20dp"
app:layout_constraintTop_toBottomOf="@id/edtDetalleProblema"
app:layout_constraintStart_toStartOf="
parent"            app:layout_constraintEnd_toEndOf="
parent" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

3.3. IMPLEMENTACIÓN DE EVENTOS CLICK Y NAVEGACIÓN ENTRE PANTALLAS (INTENTS)

Una interfaz estática no es de utilidad. El **Técnico** debe escribir la lógica en el archivo de código fuente (Kotlin) para dotar de vida a la aplicación. Para capturar la interacción con los botones se utiliza el método `setOnClickListener`. Para navegar de una pantalla a otra, el sistema operativo Android utiliza objetos llamados **Intents** (Intenciones).

Tipos de Intents:

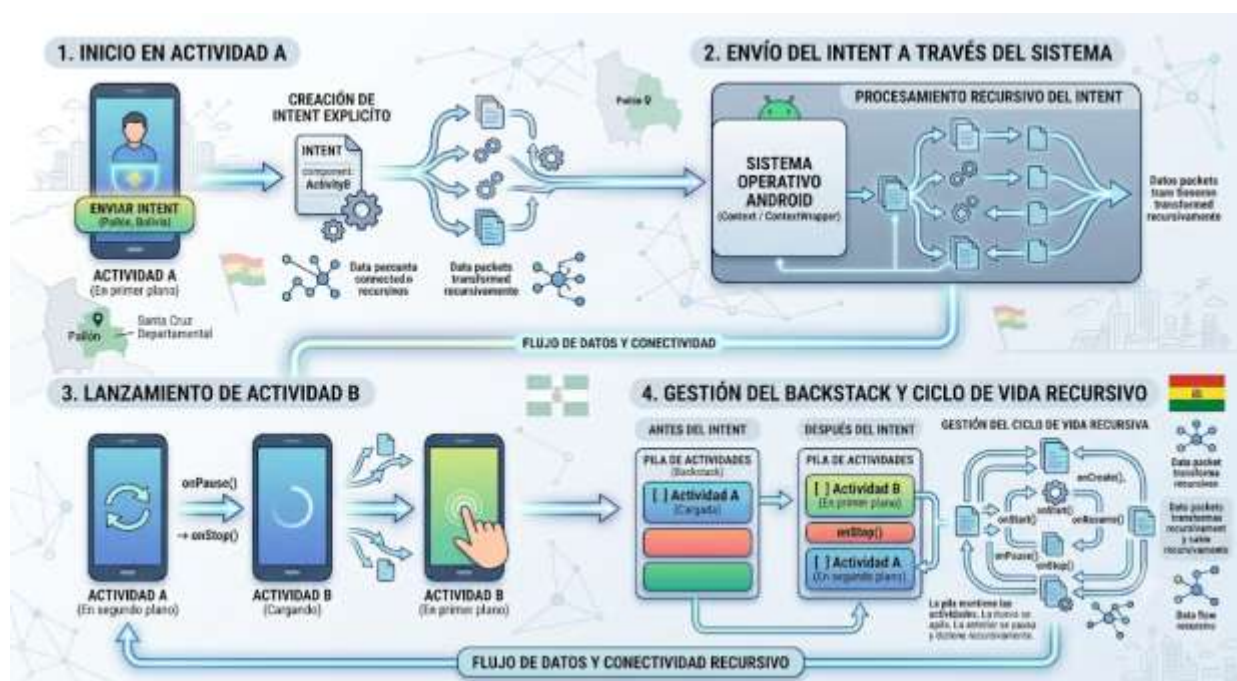
1. **Intent Explícito:** Se usa cuando sabemos exactamente a qué pantalla (Activity) queremos ir dentro de nuestra propia aplicación.



2. **Intent Implícito:** Se utiliza cuando delegamos la acción a otra aplicación instalada en el teléfono (por ejemplo, abrir Google Maps para ubicar una calle en el Barrio Ovidio Barberi o abrir el marcador telefónico para llamar a emergencias).

Kotlin

```
package com.cea.pailonconectadoimport android.content.Intentimport android.os.Bundleimport android.widget.Buttonimport android.widget.EditTextimport android.widget.Toastimport androidx.appcompat.app.AppCompatActivityclass MainActivity : AppCompatActivity() {  
    // Método de ciclo de vida que se ejecuta al crearse la pantalla override fun  
    onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState) setContentView(R.layout.activity_main)  
        // Vincula el código Kotlin con el diseño XML // Instanciación de  
        componentes mediante sus IDs definidos en el XML val btnNavegar =  
        findViewById<Button>(R.id.btnEnviarReporte) val edtDetalle =  
        findViewById<EditText>(R.id.edtDetalleProblema) // Asignación del evento  
        de escucha de clics para el botón btnNavegar.setOnClickListener {  
            val mensajeText = edtDetalle.text.toString().trim() // Validación de  
            campo vacío para evitar el envío de datos erróneos if  
            (mensajeText.isEmpty()) {  
                // Alerta rápida si el estudiante o vecino olvidó rellenar el campo  
                Toast.makeText(this, "  
                Por favor, detalle la situación del barrio", Toast.LENGTH_SHORT).show()  
            }  
            else {  
                // Creación de un Intent Explícito para saltar a la pantalla de confirmación  
                val intentExplicito = Intent(this, ConfirmationActivity::class.java)  
                // Adjuntamos información complementaria clave para la siguiente pantalla  
                intentExplicito.putExtra("  
                DATO_REPORT", mensajeText) // Iniciamos el viaje  
                hacia la nueva actividad startActivity(intentExplicito)  
            }  
        }  
    }  
}
```



3.4. INSERCIÓN DE ÍCONOS, TIPOGRAFÍAS Y RECURSOS MULTIMEDIA LOCALES

Para que un vecino de barrios como **María Belén** o **San Antonio** se sienta familiarizado con la aplicación, el diseño visual debe ser impecable y contextualizado. Para ello, gestionaremos los recursos multimedia de forma local dentro del directorio res/ del proyecto.

Organización de Recursos Comunes:

1. **res/drawable:** Almacena imágenes PNG/JPEG y gráficos vectoriales en formato XML (Vector Assets). Es altamente recomendable usar vectores para los íconos (como bicicletas, herramientas de limpieza, alimentos), ya que no pierden resolución ni aumentan el peso de la app.
2. **res/font:** Carpeta dedicada a almacenar fuentes tipográficas personalizadas (archivos .ttf o .otf). Nos permite desmarcarnos de las fuentes genéricas del sistema y aplicar tipografías con excelente legibilidad.

Pasos para aplicar una fuente local a toda la interfaz:

1. Descargar el archivo de la fuente (por ejemplo, *Poppins-Regular.ttf*).



2. Pegarlo dentro de res/font/ cuidando usar letras minúsculas y guiones bajos únicamente (poppins_regular.xml).
3. Aplicarlo en el XML mediante el atributo android:fontFamily="@font/poppins_regular".

XML

```
<TextView    android:id="@+id/txtBienvenidaCea"    android:layout_width="
wrap_content"    android:layout_height="
wrap_content"    android:text="
Bienvenidos al Módulo Tecnológico Barrio Saó"
android:fontFamily="@font/poppins_bold" android:textSize="18sp"
android:textColor="
#0D47A1" android:gravity="
center" />
```

3.5. INTERACCIÓN CON ALERTAS (TOASTS, SNACKBARS) Y DIÁLOGOS

Un principio dorado de la UX es: *"Mantén siempre informado al usuario de lo que ocurre en el sistema"*. Si un usuario del **Barrio Jardines de Pailón** presiona un botón y la pantalla no reacciona, pensará que el teléfono se colgó o que la app no sirve.

Herramientas de Retroalimentación Visual:

Tipo de Alerta	Duración en Pantalla	Nivel de Interrupción	Caso de Uso Ideal en Pailón
Toast	Corta (2-3 segundos)	Muy Bajo (no interrumpe)	Avisar que un formulario se guardó exitosamente de forma local.
Snackbar	Programable (con opción a botón de acción)	Bajo (aparece en la base)	Confirmar envío de reporte, permitiendo "Deshacer" si el usuario se equivocó.
AlertDialog	Persistente (requiere acción explícita)	Alto (bloquea la pantalla entera)	Confirmar si el transportista desea cerrar sesión o borrar un registro crítico.



Kotlin

```
// 1. IMPLEMENTACIÓN DE UN SNACKBAR (Requiere una vista contenedora como base)
val miContenedor =
findViewById<androidx.constraintlayout.widget.ConstraintLayout>(R.id.layoutRaiz)
Snackbar.make(miContenedor, "
Reporte enviado para Barrio Las Misiones", Snackbar.LENGTH_LONG) .setAction("
DESHACER") {
// Código para cancelar la operación si el usuario presionó por accidente
Toast.makeText(this, "
Acción cancelada", Toast.LENGTH_SHORT).show() }
.show()
// 2. IMPLEMENTACIÓN DE UN ALERT DIALOG (Para decisiones trascendentales)
val constructorDialogo = androidx.appcompat.app.AlertDialog.Builder(this)
constructorDialogo.setTitle("
Confirmación Requerida")
constructorDialogo.setMessage("¿Desea reservar el coliseo del Barrio 27 de Mayo
para las olimpiadas del CEA?")
constructorDialogo.setPositiveButton("
Sí, Reservar") {
_, _ -> Toast.makeText(this, "
Reserva procesada exitosamente", Toast.LENGTH_SHORT).show()}
constructorDialogo.setNegativeButton("
Cancelar", null) // Cierra la ventana sin realizar ninguna
acciónconstructorDialogo.create().show()
```

INTERFAZ DE USUARIO CON MÚLTIPLES NOTIFICACIONES			
COMPARACIÓN DE COMPONENTES DE NOTIFICACIÓN DE UI			
TIPO	1 TOAST (Parte Inferior)	2 SNACKBAR (Con Acción)	3 ALERTDIALOG (Centrado)
UBICACIÓN			
DURACIÓN	Características: • Ubicación inferior • Duración breve • No interactivo • Simple (Texto)	Características: • Ubicación inferior • Duración media (5-10s) • Interactividad opcional (Botón) • Texto y Acción	Características: • Ubicación central (Modal) • Duración hasta respuesta • Interactividad obligatoria • Título, Texto y Botones (Confirmación/Cancelación)
INTERACTIVIDAD			
PROPÓSITO			



3.6. PRINCIPIOS DE ACCESIBILIDAD MÓVIL Y DISEÑO INCLUSIVO PARA ENTORNOS DE CONECTIVIDAD LIMITADA

En municipios como Pailón, las condiciones tecnológicas reales difieren sustancialmente de las grandes capitales. Muchos de nuestros usuarios consumen datos móviles prepago de forma dosificada y se desplazan por zonas con cobertura inestable (como los barrios periféricos **San José** o **Ovidio Barberi**). El rol del **Técnico** moderno es diseñar pensando en estas limitaciones.

Pautas de Inclusión y Optimización:

1. **Diseño de Alto Contraste para Exteriores:** Los mototaxistas y comerciantes trabajan bajo la luz directa del sol. Las interfaces deben usar contrastes marcados (textos oscuros sobre fondos completamente claros o viceversa). Eviten usar grises claros sobre fondos blancos, ya que la pantalla se vuelve invisible bajo el sol.
2. **ContentDescription en Imágenes:** Toda imagen no decorativa debe llevar este atributo en el XML. Esto permite que los lectores de pantalla (como TalkBack) lean en voz alta la descripción de la vista a personas con discapacidad visual total o parcial.
3. **UI Resiliente sin Conexión (Offline-First):** Si la aplicación está intentando enviar un reporte desde el **Barrio San Expedito** y la señal cae, la interfaz no debe colapsar con un error técnico incomprensible. Debe mostrar un estado amigable: *"Guardado localmente. Se enviará de forma automática en cuanto recuperes señal"*.

3.7. PATRONES DE DISEÑO MODERNOS (MATERIAL DESIGN 3) Y CONSISTENCIA VISUAL EN APLICACIONES ANDROID

Material Design 3 es la guía de diseño de código abierto creada por Google para construir interfaces Android modernas y consistentes. No se trata simplemente de colores atractivos, sino de un sistema lógico de componentes y comportamientos que los usuarios ya reconocen de forma intuitiva por usar aplicaciones populares como Gmail o WhatsApp.



Pilares de la consistencia visual aplicados a nuestro proyecto:

1. **Colores Dinámicos y Significado:** Material Design define roles específicos para el color. El color *Primary* se usa para los botones principales y marcas de identidad de la app; el color *Error* (tonalidades rojas) se reserva exclusivamente para validaciones fallidas y estados críticos.
2. **Zonas de Interacción Cómodas (Touch Targets):** Cualquier elemento interactivo (botones, selectores, íconos de menú) debe tener un área táctil mínima de **48x48 dp**. Si un botón es más pequeño, a los usuarios con dedos grandes o transportistas en movimiento les costará presionarlo, restándole usabilidad a la aplicación.
3. **Jerarquía Tipográfica Clara:** Establece tamaños y pesos visuales estrictos. Los títulos principales se destacan con fuentes gruesas (*Bold*), mientras que los contenidos informativos secundarios utilizan variantes normales (*Regular*) y tamaños reducidos para no saturar visualmente la pantalla.

3.8. EL IMPACTO DE LA INTELIGENCIA ARTIFICIAL EN EL PROTOTIPADO RÁPIDO DE INTERFACES (UI/UX) Y DESARROLLO FRONTEND MÓVIL

La Inteligencia Artificial ha transformado por completo la forma en que los ingenieros y **Técnicos** diseñan y programan aplicaciones móviles. En lugar de pasar semanas dibujando pantallas desde cero, hoy un desarrollador puede acelerar sustancialmente su flujo de trabajo frontend utilizando herramientas automatizadas de IA.

Aplicación de la IA en la labor del Técnico:

1. **Generación de Maquetas (Wireframes) Asistida:** Plataformas impulsadas por IA permiten transcribir una idea en texto plano (por ejemplo: "Genera una pantalla de catálogo de productos agrícolas para el mercado de Pailón con Material Design 3") y obtener un esqueleto visual editable en cuestión de segundos.
2. **Optimización de Código XML Basada en Contexto:** Herramientas de IA ayudan a auditar el código fuente XML escrito por el programador para detectar errores de accesibilidad,



sugerir la eliminación de vistas innecesarias que ralentizan el teléfono y automatizar la creación de restricciones complejas dentro de un ConstraintLayout.

3. **Creación de Textos (UX Writing) Empáticos:** La IA puede calibrar los mensajes de error y las notificaciones dentro de la app para que utilicen un tono adecuado, comprensible y cercano al contexto sociocultural de la población local, evitando tecnicismos informáticos fríos que asusten al usuario.

CIERRE TEÓRICO OBLIGATORIO

❑ *Mitos vs. Realidades*

Mito Técnico Común	Realidad Comprobada de la UX
"El diseño de interfaces (UI) solo consiste en hacer que la aplicación móvil luzca bonita, colorida y llamativa."	La UI estructura la interacción lógica. Una app hermosa es inútil si los botones clave no se encuentran fácilmente o confunden al usuario.
"Para que una aplicación móvil se adapte a todas las pantallas, basta con crear muchas carpetas de layouts con distintos tamaños."	Eso satura el tamaño del instalador. La solución real es programar de forma fluida usando restricciones inteligentes con ConstraintLayout.

❑ *Glosario de Términos*

1. **UX (User Experience):** Disciplina enfocada en analizar cómo se siente un usuario al usar un producto, priorizando la facilidad, la velocidad y la satisfacción general durante el proceso.
2. **UI (User Interface):** Conjunto de elementos visuales, pantallas, botones, íconos y tipografías con los cuales las personas interactúan físicamente en un dispositivo digital.
3. **DP (Density-independent Pixels):** Unidad de medida virtual abstracta que utiliza Android para garantizar que los elementos de la interfaz mantengan el mismo tamaño físico en pantallas con distintas densidades de píxeles.
4. **SP (Scale-independent Pixels):** Unidad de medida idéntica al dp, pero calibrada específicamente para fuentes de texto, la cual respeta de forma automática las preferencias de tamaño configuradas por el usuario en el sistema.



5. **Vector Asset:** Gráfico vectorial basado en coordenadas matemáticas XML en lugar de mapas de bits (píxeles), permitiendo un escalado infinito sin pérdida alguna de nitidez y con un peso en bytes mínimo.

□ **Ponte a Prueba**

1. **Si deseas diseñar una interfaz que no contenga contenedores anidados y distribuya sus componentes mediante anclas flexibles para adaptarse a cualquier pantalla, ¿qué Layout debes elegir obligatoriamente?**

1. A) RelativeLayout
2. B) ConstraintLayout
3. C) LinearLayout Horizontal

2. **¿Cuál es el tamaño mínimo recomendado por las guías de accesibilidad de Material Design para cualquier componente interactivo (Touch Target) dentro de la app?**

1. A) 24x24 dp
2. B) 100x100 px
3. C) 48x48 dp

3. **¿Qué componente de retroalimentación visual debes programar si necesitas detener por completo la navegación del usuario hasta que confirme explícitamente si desea eliminar una reserva del Coliseo Municipal en el Barrio 27 de Mayo?**

1. A) Un AlertDialog personalizado
2. B) Un Snackbar con acción extendida
3. C) Un Toast de duración larga

(Respuestas correctas: 1=B, 2=C, 3=A)



VALORACIÓN



IMPACTO SOCIAL Y COMUNITARIO

El diseño de interfaces no es únicamente una tarea técnica de programación; es un acto de responsabilidad social profunda. Cuando un **Técnico en Sistemas Informáticos** diseña una interfaz confusa o que exige teléfonos costosos de última generación, está excluyendo de forma directa a los sectores más vulnerables de nuestra población. En Pailón, una aplicación mal diseñada para el **CEA Herman Ortiz Camargo** significa que una madre de familia de comunidades alejadas o un agricultor del Barrio Ovidio Barberi no podrán acceder a convocatorias de capacitación, no lograrán reportar una emergencia vecinal o perderán oportunidades comerciales valiosas. Diseñar con empatía, inclusión y accesibilidad web democratiza el acceso a la tecnología y unifica el tejido social de nuestro municipio.

IMPACTO ÉTICO, PRIVACIDAD Y CONECTIVIDAD LOCAL

Desde la perspectiva de la ética profesional, la recopilación de datos a través de interfaces móviles debe ser transparente y respetuosa. Diseñar pantallas que obliguen al rastreo continuo de la ubicación GPS para mapear los barrios de Pailón (como **San Antonio** o **Las Misiones**) compromete la privacidad de nuestros vecinos de forma innecesaria. Las interfaces deben solicitar permisos explícitos solo cuando sea estrictamente indispensable para el servicio.

SOSTENIBILIDAD Y MITIGACIÓN DEL E-WASTE

En el ámbito medioambiental, el desarrollo frontend irresponsable acelera el fenómeno del **e-waste** (chatarra electrónica). Si programamos interfaces pesadas que demandan procesadores de alta gama y gran cantidad de memoria RAM para renderizar animaciones innecesarias, obligamos indirectamente a los usuarios locales a desechar sus teléfonos antiguos que aún funcionan perfectamente bien. Al optimizar los layouts, emplear gráficos vectoriales locales livianos y reducir el procesamiento innecesario de la UI, prolongamos la vida útil de los dispositivos móviles en los hogares de Pailón, mitigando la contaminación por residuos tecnológicos y promoviendo una cultura de consumo digital sostenible.



PRODUCCIÓN



RETO FINAL: LABORATORIO PRÁCTICO PASO A PASO

Nombre del Proyecto: *App de Alertas Vecinales CEA - Pailón*

Objetivo: Desarrollar una aplicación móvil interactiva con dos pantallas que permita a los estudiantes del CEA reportar necesidades de mantenimiento en el nuevo módulo educativo (Barrio Saó) o incidencias en los barrios colindantes, implementando ConstraintLayout, eventos Click, envío de datos a través de un Intent Explícito y retroalimentación mediante un Snackbar dinámico.

Paso 1: Configurar el archivo de interfaz principal (activity_main.xml)

Crea un nuevo proyecto en Android Studio con una actividad vacía (*Empty Views Activity*) y sustituye el código del diseño de la primera pantalla por la siguiente estructura robusta de restricciones:

XML

```
<?xml version="1.0" encoding="
utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout      xmlns:android="
http:
//schemas.android.com/apk/res/android"      xmlns:app="
http:
//schemas.android.com/apk/res-auto"      android:id="@+id/layoutPrincipal"
android:layout_width="
match_parent"      android:layout_height="
match_parent"      android:padding="16dp"      android:background="
#FAFAFA"> <TextView      android:id="@+id/lblTituloApp"
android:layout_width="
wrap_content"      android:layout_height="
wrap_content"      android:text="
SISTEMA DE ALERTAS PAILÓN"      android:textSize="20sp"
android:textStyle="
bold"      android:textColor="
#1565C0"      android:layout_marginTop="8dp"
app:layout_constraintTop_toTopOf="
parent"      app:layout_constraintStart_toStartOf="
```



```

parent"          app:layout_constraintEnd_toEndOf="
parent" />      <TextView          android:id="@+id/lblSubtituloApp"
android:layout_width="0dp"          android:layout_height="
wrap_content"          android:text="
Reporte incidencias desde Barrio Saó, Fátima, 24 de Septiembre y más."
android:textSize="12sp"          android:textColor="
#555555"          android:gravity="
center"          android:layout_marginTop="6dp"
app:layout_constraintTop_toBottomOf="@id/lblTituloApp"
app:layout_constraintStart_toStartOf="
parent"          app:layout_constraintEnd_toEndOf="
parent" />      <EditText          android:id="@+id/txtNombreBarrio"
android:layout_width="0dp"          android:layout_height="
wrap_content"          android:hint="
Nombre del Barrio (Ej. Barrio Central Fátima)"          android:inputType="
textCapWords" android:minHeight="48dp" android:layout_marginTop="24dp"
app:layout_constraintTop_toBottomOf="@id/lblSubtituloApp"
app:layout_constraintStart_of="
parent"          app:layout_constraintStart_toStartOf="
parent"          app:layout_constraintEnd_toEndOf="
parent" />      <EditText          android:id="@+id/txtDescripcionReporte"
android:layout_width="0dp"          android:layout_height="
wrap_content"          android:hint="
Describe el problema detalladamente..."          android:inputType="
textMultiLine"          android:minHeight="48dp"
android:layout_marginTop="12dp"
app:layout_constraintTop_toBottomOf="@id/txtNombreBarrio"
app:layout_constraintStart_toStartOf="
parent"          app:layout_constraintEnd_toEndOf="
parent" />      <Button          android:id="@+id/btnProcesarAlerta"
android:layout_width="0dp"          android:layout_height="
wrap_content"          android:text="
PROCESAR ALERTA COMUNITARIA"          android:backgroundTint="
#1565C0"          android:textColor="
#FFFFFF"          android:textSize="14sp"          android:textStyle="
bold"          android:layout_marginTop="24dp"          android:padding="12dp"
app:layout_constraintTop_toBottomOf="@id/txtDescripcionReporte"
app:layout_constraintStart_toStartOf="
parent"          app:layout_constraintEnd_toEndOf="
parent" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

Paso 2: Configurar la interfaz de la pantalla secundaria (activity_detail.xml)

Crea una nueva actividad llamada DetailActivity e implementa este XML limpio diseñado exclusivamente para mostrar los resultados procesados:



XML

```
<?xml version="1.0" encoding="
utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout      xmlns:android="
http:
//schemas.android.com/apk/res/android"      xmlns:app="
http:
//schemas.android.com/apk/res-auto"      android:layout_width="
match_parent"      android:layout_height="
match_parent"      android:padding="20dp"      android:background="
#FFFFFF">      <TextView      android:id="@+id/lblDetalleTitulo"
android:layout_width="
wrap_content"      android:layout_height="
wrap_content"      android:text="
Resumen del Reporte Recibido"      android:textSize="18sp"
android:textStyle="
bold"      android:textColor="
#2E7D32" app:layout_constraintTop_toTopOf="
parent"      app:layout_constraintStart_toStartOf="
parent"      app:layout_constraintEnd_toEndOf="
parent" />      <TextView      android:id="@+id/lblResultadoBarrio"
android:layout_width="0dp"      android:layout_height="
wrap_content"      android:text="
Barrio: Cargando..."      android:textSize="15sp"      android:textStyle="
bold"      android:textColor="
#333333"      android:layout_marginTop="32dp"
app:layout_constraintTop_toBottomOf="@id/lblDetalleTitulo"
app:layout_constraintStart_toStartOf="
parent"      app:layout_constraintEnd_toEndOf="
parent" />      <TextView      android:id="@+id/lblResultadoDescripcion"
android:layout_width="0dp"      android:layout_height="
wrap_content"      android:text="
Detalle: Cargando..."      android:textSize="14sp"      android:textColor="
#666666"      android:layout_marginTop="12dp"
app:layout_constraintTop_toBottomOf="@id/lblResultadoBarrio"
app:layout_constraintStart_toStartOf="
parent"      app:layout_constraintEnd_toEndOf="
parent" />      <Button      android:id="@+id/btnVolverAtras"
android:layout_width="0dp"      android:layout_height="
wrap_content"      android:text="
REGRESAR AL INICIO"      android:backgroundTint="
#CCCCCC"      android:textColor="
#333333"      android:layout_marginBottom="16dp"
app:layout_constraintBottom_toBottomOf="
parent"      app:layout_constraintStart_toStartOf="
parent"      app:layout_constraintEnd_toEndOf="
```



```
parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Paso 3: Programar el archivo lógico principal (MainActivity.kt)

Abre tu archivo Kotlin principal y codifica el control de eventos del formulario, las validaciones obligatorias y la lógica de empaquetado del Intent Explícito:

Kotlin

```
package com.cea.pailonconectadoimport android.content.Intentimport
android.os.Bundleimport android.widget.Buttonimport
android.widget.EditTextimport androidx.appcompat.app.AppCompatActivityimport
androidx.constraintlayout.widget.ConstraintLayoutimport
com.google.android.material.snackbar.Snackbarclass MainActivity :
AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState) setContentView(R.layout.activity_main)
        // Instanciación manual de las vistas declaradas en el archivo XML principal
        val contenedorRaiz = findViewById<ConstraintLayout>(R.id.layoutPrincipal)
        val campoBarrio = findViewById<EditText>(R.id.txtNombreBarrio) val
        campoDescripcion = findViewById<EditText>(R.id.txtDescripcionReporte) val
        botonProcesar = findViewById<Button>(R.id.btnProcesarAlerta) // Escucha
        activa de interacciones en el botón de procesamiento
        botonProcesar.setOnClickListener {
            val textoBarrio = campoBarrio.text.toString().trim() val
            textoDescripcion = campoDescripcion.text.toString().trim() // Control
            de flujos vacíos para robustez de La Experiencia de Usuario if
            (textoBarrio.isEmpty() || textoDescripcion.isEmpty()) {
                // Notificación no intrusiva solicitando la corrección inmediata de los datos
                Snackbar.make(contenedorRaiz, "
                Complete todos los campos del formulario", Snackbar.LENGTH_LONG).show()
            }
            else {
                // Configuración detallada del Intent Explícito apuntando a DetailActivity
                val intentNavegacion = Intent(this, DetailActivity::class.java)
                // Inyección de parámetros estructurados mediante claves string estables
                intentNavegacion.putExtra("
                KEY_BARRIO", textoBarrio) intentNavegacion.putExtra("
                KEY_DESCRIPCION", textoDescripcion) // Disparo de
                la navegación hacia la pantalla de destino
                startActivity(intentNavegacion)
            }
        }
    }
}
```




Paso 4: Programar la lógica de la pantalla secundaria (DetailActivity.kt)

Escribe la lógica necesaria en la segunda actividad para desempaquetar la información transportada por el Intent y renderizarla de manera dinámica en los componentes visuales:

Kotlin

```
package com.cea.pailonconectadoimport android.os.Bundleimport
android.widget.Buttonimport android.widget.TextViewimport
androidx.appcompat.app.AppCompatActivityclass DetailActivity :
AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_detail)           // Vinculación de Los
        componentes dinámicos de texto y control de la actividad           val vistaBarrio
        = findViewById<TextView>(R.id.lblResultadoBarrio)           val vistaDescripcion =
        findViewById<TextView>(R.id.lblResultadoDescripcion)           val botonVolver =
        findViewById<Button>(R.id.btnVolverAtras)           // Extracción segura del bundle
        de datos transportados en el Intent de origen           val datosRecibidos =
        intent.extras           if (datosRecibidos != null) {
            // Recuperación de los datos mediante el mapeo exacto de claves string
            val barrioExtraido = datosRecibidos.getString("
            KEY_BARRIO", "
            Barrio Desconocido")           val descripcionExtraida =
            datosRecibidos.getString("
            KEY_DESCRIPCION", "
            Sin detalles disponibles")           // Actualización visual de los componentes
            en pantalla con los datos reales           vistaBarrio.text = "
            Barrio reportado: barrioExtraido"           vistaDescripcion.text = "
            Detalle técnico: descripcionExtraida"           }
            // Finalización controlada del ciclo de vida de la actividad para regresar al
            inicio           botonVolver.setOnClickListener {
            finish() // Cierra de forma limpia la pantalla actual y vuelve a la anterior en
            la pila           }
        }
    }
}
```

RECURSOS ADICIONALES

1. **Documentación Oficial de Android Developers - Layouts:** Guía completa del equipo de Google para dominar en profundidad el comportamiento y los atributos avanzados de ConstraintLayout y LinearLayout.
<https://developer.android.com/guide/topics/ui/declaring-layout>



2. **Guía Oficial de Material Design 3:** Repositorio de diseño interactivo de Google que detalla los estándares de paletas de colores, tipografías, accesibilidad y componentes para interfaces móviles modernas. <https://m3.material.io>
3. **Curso Práctico de Accesibilidad en Android (Google Codelabs):** Laboratorio guiado paso a paso para aprender a auditar interfaces móviles, añadir descripciones correctas de componentes y adaptar aplicaciones para personas de la tercera edad o con capacidades diferenciadas. <https://developer.android.com/codelabs/android-accessibility>